

<i>Trường ĐH Công Thương TP.HCM</i> Khoa: CNTT Bộ môn: Kỹ thuật phần mềm LẬP TRÌNH MÃ NGUỒN MỞ	BÀI 5 TƯƠNG TÁC CƠ SỞ DỮ LIỆU VỚI MYSQL VÀ PDO Làm việc với Cơ sở dữ liệu MySQL (Phần nâng cao)	
---	---	---

A. MỤC TIÊU

Sau khi hoàn thành bài lab này, sinh viên sẽ:

- Biết cách sử dụng các hàm tổng hợp (Aggregate Functions) và GROUP BY, HAVING trong MySQL.
- Thực hiện được truy vấn nhiều bảng (JOIN).
- Sử dụng Subquery (truy vấn con) trong các tình huống thực tế.
- Viết truy vấn nâng cao kết hợp điều kiện lọc dữ liệu.
- Ứng dụng kiến thức để giải quyết các yêu cầu phức tạp hơn từ bài toán thực tế.

B. NỘI DUNG THỰC HÀNH

1. Cơ sở lý thuyết

- Hàm tổng hợp trong MySQL
 - COUNT() – đếm số bản ghi.
 - SUM() – tính tổng.
 - AVG() – tính trung bình.
 - MIN() và MAX() – tìm giá trị nhỏ nhất/lớn nhất.
- GROUP BY & HAVING
 - GROUP BY dùng để nhóm dữ liệu theo một hoặc nhiều cột.
 - HAVING dùng để lọc kết quả sau khi nhóm.
- JOIN
 - INNER JOIN: chỉ lấy bản ghi khớp ở cả hai bảng.
 - LEFT JOIN: lấy tất cả bản ghi bảng bên trái và dữ liệu khớp từ bảng bên phải.
 - RIGHT JOIN: ngược lại với LEFT JOIN.
 - CROSS JOIN: kết hợp mọi bản ghi của hai bảng.
- Subquery
 - Truy vấn con trong WHERE hoặc FROM.

- Dùng để lọc, so sánh, hoặc làm nguồn dữ liệu cho truy vấn chính.
- Kết hợp các câu lệnh nâng cao
 - Dùng JOIN + GROUP BY.
 - Dùng Subquery + HAVING.
 - Truy vấn nhiều điều kiện.

2. Bài tập tại lớp

Sinh viên phải thực hiện các bài tập bằng file PHP và hiển thị kết quả dưới dạng HTML đơn giản.

Tạo cơ sở dữ liệu bao gồm các bảng chính:

- products (product_id, name, price, category_id)
- categories (category_id, category_name)
- orders (order_id, order_date, customer_id)
- order_details (order_id, product_id, quantity, price)
- customers (customer_id, name, email)

```
-- Xóa CSDL nếu đã tồn tại
DROP DATABASE IF EXISTS lab3_shop;

-- Tạo mới CSDL
CREATE DATABASE lab3_shop CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Sử dụng CSDL
USE lab3_shop;

-- Bảng categories
CREATE TABLE categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL
);

-- Bảng products
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    category_id INT,
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

```

-- Bảng customers
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE
);

-- Bảng orders
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    order_date DATE NOT NULL,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Bảng order_details
CREATE TABLE order_details (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Dữ liệu mẫu cho categories
INSERT INTO categories (category_name) VALUES
('Điện thoại'),
('Laptop'),
('Phụ kiện');

-- Dữ liệu mẫu cho products
INSERT INTO products (name, price, category_id) VALUES
('iPhone 15 Pro Max', 33990000, 1),
('Samsung Galaxy S24 Ultra', 28990000, 1),
('MacBook Air M2', 28990000, 2),
('Dell XPS 13', 26990000, 2),
('Tai nghe AirPods Pro 2', 5490000, 3),
('Chuột Logitech MX Master 3S', 2490000, 3),
('Cáp sạc USB-C', 199000, 3),
('iPad Pro 12.9', 30990000, 1),
('Asus ZenBook', 18990000, 2),

```

```

('Bàn phím cơ Keychron K2', 2390000, 3);

-- Dữ liệu mẫu cho customers
INSERT INTO customers (name, email) VALUES
('Nguyễn Văn A', 'a.nguyen@example.com'),
('Trần Thị B', 'b.tran@example.com'),
('Lê Văn C', 'c.le@example.com');

-- Dữ liệu mẫu cho orders
INSERT INTO orders (order_date, customer_id) VALUES
('2025-08-01', 1),
('2025-08-02', 2),
('2025-08-03', 3),
('2025-08-05', 1);

-- Dữ liệu mẫu cho order_details
INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 33990000),
(1, 5, 2, 5490000),
(2, 3, 1, 28990000),
(3, 2, 1, 28990000),
(3, 6, 1, 2490000),
(4, 4, 1, 26990000),
(4, 7, 3, 199000);

```

Bài tập 01: Thống kê số lượng sản phẩm trong từng loại hàng

Yêu cầu:

- Hiển thị category_name và số lượng sản phẩm (COUNT) trong mỗi loại.

Hướng dẫn:

```

SELECT c.category_name, COUNT(p.product_id) AS total_products
FROM categories c
LEFT JOIN products p ON c.category_id = p.category_id
GROUP BY c.category_name;

```

Giải thích:

- LEFT JOIN để đảm bảo loại hàng không có sản phẩm vẫn hiện lên.
- GROUP BY nhóm theo tên loại hàng.

Bài tập 02: Tính tổng doanh thu từng ngày

Yêu cầu:

- Hiển thị order_date và tổng doanh thu (sum(quantity × price)).

Hướng dẫn:

```
SELECT o.order_date, SUM(od.quantity * od.price) AS total_revenue
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
GROUP BY o.order_date;
```

Giải thích:

- SUM(od.quantity * od.price) tính tổng tiền theo ngày.
- GROUP BY o.order_date nhóm dữ liệu theo ngày đặt hàng.

Bài tập 03: Tìm loại hàng có trên 5 sản phẩm

Yêu cầu:

- Hiển thị category_name và số lượng sản phẩm, nhưng chỉ hiện loại có > 5 sản phẩm.

Hướng dẫn:

```
SELECT c.category_name, COUNT(p.product_id) AS total_products
FROM categories c
JOIN products p ON c.category_id = p.category_id
GROUP BY c.category_name
HAVING COUNT(p.product_id) > 5;
```

Giải thích:

- HAVING dùng để lọc sau khi nhóm dữ liệu.

Bài tập 04: Danh sách khách hàng và tổng tiền đã mua

Yêu cầu:

- Hiển thị customer_id, name, và tổng số tiền đã mua.
- Chỉ hiện khách hàng có tổng chi tiêu > 1.000.000.

Hướng dẫn:

```
SELECT c.customer_id, c.name, SUM(od.quantity * od.price) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_details od ON o.order_id = od.order_id
GROUP BY c.customer_id, c.name
HAVING total_spent > 1000000;
```

Giải thích:

- Dùng GROUP BY kết hợp nhiều cột (customer_id, name).
- Dùng HAVING để lọc kết quả.

Bài tập 05: Tìm sản phẩm có giá cao nhất trong từng loại hàng

Yêu cầu:

- Hiển thị category_name, name của sản phẩm đắt nhất.

Hướng dẫn:

```
SELECT c.category_name, p.name, p.price
FROM products p
JOIN categories c ON p.category_id = c.category_id
WHERE p.price = (
    SELECT MAX(p2.price)
    FROM products p2
    WHERE p2.category_id = p.category_id
);
```

Giải thích:

- Subquery dùng để tìm giá cao nhất cho từng loại.
- So sánh giá của sản phẩm hiện tại với giá cao nhất.

Bài tập 06: Liệt kê sản phẩm chưa từng được đặt hàng

Yêu cầu:

- Hiển thị product_id, name của sản phẩm chưa có trong order_details.

Hướng dẫn:

```
SELECT p.product_id, p.name
FROM products p
LEFT JOIN order_details od ON p.product_id = od.product_id
WHERE od.product_id IS NULL;
```

Giải thích:

- LEFT JOIN + IS NULL để tìm bản ghi không tồn tại ở bảng bên phải.

Bài tập 07: Khách hàng mua nhiều sản phẩm nhất

Yêu cầu:

- Hiển thị thông tin khách hàng có tổng số lượng sản phẩm mua nhiều nhất.

Hướng dẫn:

```
SELECT c.customer_id, c.name, SUM(od.quantity) AS total_items
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_details od ON o.order_id = od.order_id
GROUP BY c.customer_id, c.name
ORDER BY total_items DESC
LIMIT 1;
```

Giải thích:

- ORDER BY ... DESC để sắp xếp giảm dần.
- LIMIT 1 lấy duy nhất bản ghi đầu tiên.

3. Bài tập ôn luyện

Bài tập 08: Thống kê tổng số lượng và doanh thu của từng loại sản phẩm.

Bài tập 09: Tìm 3 sản phẩm bán chạy nhất (theo số lượng bán ra).

Bài tập 10: Liệt kê 5 khách hàng chi tiêu nhiều nhất.

Bài tập 11: Tìm loại hàng có doanh thu cao nhất.

Bài tập 12: Liệt kê tất cả sản phẩm và số lần được đặt hàng (nếu chưa đặt thì là 0).